

Report: Approach to Building a Basic URL Shortener Application

1. Introduction

The objective of this project was to design and implement a **basic URL Shortener application** using Python and Flask. The application allows users to submit a long URL, generates a short and unique code for that URL, stores the mapping in a database, and redirects users to the original URL when the shortened link is accessed. Additionally, the application maintains a history page where all previously shortened URLs can be viewed.

This report explains, in detail, the **thought process, design decisions, technology stack, and implementation approach** used to build the solution. It also discusses challenges faced and possible future improvements.

2. Problem Understanding and Requirements

Before starting development, I analyzed the core requirements of a basic URL shortener:

1. Accept a long (original) URL from the user.
2. Validate whether the URL is in a correct format.
3. Generate a short, unique identifier (short code).
4. Store the mapping between the original URL and short code.
5. Redirect users from the short URL to the original URL.
6. Provide a simple UI for interaction.
7. Maintain a history of shortened URLs.

The scope was intentionally kept minimal ("Basic Version") to focus on clarity, correctness, and clean backend logic rather than advanced features like authentication or analytics.

3. Technology Stack and Justification

The following technologies were selected:

3.1 Flask

Flask was chosen as the backend framework because:

- It is lightweight and easy to set up.
- It is well-suited for small to medium web applications.
- It allows rapid prototyping with minimal boilerplate.

3.2 Flask-SQLAlchemy

Flask-SQLAlchemy was used as the ORM layer to:

- Simplify database interactions.
- Abstract raw SQL queries into Python objects.
- Improve code readability and maintainability.

3.3 SQLite

SQLite was selected as the database because:

- It requires no external server.
- It is ideal for small-scale and prototype applications.
- It integrates seamlessly with Flask-SQLAlchemy.

3.4 HTML (Jinja Templates)

HTML templates were used for the frontend:

- `index.html` for URL submission and short link display.
- `history.html` for displaying all stored URLs.

3.5 Validators Library

The `validators` library was used to ensure that the submitted URL is valid before storing it in the database.

4. Application Architecture

The application follows a **simple MVC-style structure**:

- **Model:** `models.py` defines the database schema and logic for generating short codes.
- **View:** HTML templates (`index.html`, `history.html`) handle user interaction.
- **Controller:** `app.py` manages routing, request handling, and application flow.

This separation of concerns ensures better readability and easier maintenance.

5. Database Design

5.1 URLModel

The core database table is represented by the `URLModel` class:

- `id`: Primary key (integer).
- `original_url`: Stores the full URL submitted by the user.
- `short_code`: Stores the generated unique short identifier.

The table design is intentionally minimal to support fast lookups and simplicity.

5.2 Short Code Generation

A short code is generated using:

- Uppercase letters
- Lowercase letters
- Digits

A random combination of 6 characters is created using Python's `random.choices()` function. This provides:

- A large number of possible combinations.
 - Low probability of collision in a basic application.
-

6. Backend Logic and Routing

6.1 Application Initialization

- Flask app is initialized.
- Database URI and secret key are configured.
- Database tables are created within the application context.

This ensures that the database is ready before handling requests.

6.2 Index Route (/)

This route supports both **GET** and **POST** requests:

- **GET:** Renders the homepage with a form to submit a URL.
- **POST:**
 - Accepts the user input.
 - Validates the URL.
 - Creates a new database entry.
 - Displays the generated short URL.

Flash messages are used to provide feedback in case of invalid input.

6.3 Redirect Route (**/<short_code>**)

This is the core functionality of the URL shortener:

- Extracts the **short_code** from the URL.
- Searches the database for a matching record.
- Redirects the user to the corresponding original URL.

If the short code does not exist, a 404 error is automatically returned.

6.4 History Route (**/history**)

- Fetches all URL records from the database.
 - Displays them in a tabular format.
 - Helps users view previously shortened links.
-

7. Frontend Design Approach

The frontend was kept clean and minimal:

- Simple forms for user input.
- Clear display of the generated short URL.
- A separate history page for better organization.

Jinja templating was used to dynamically pass data from Flask to HTML.

8. Error Handling and Validation

Key validation and error-handling strategies include:

- URL validation before database insertion.
- Prevention of empty submissions.
- Automatic 404 handling for invalid short codes.

These checks ensure a smoother user experience and prevent invalid data storage.

9. Testing and Verification

The application was tested by:

- Submitting valid and invalid URLs.
- Verifying database entries in SQLite.
- Testing redirection using generated short URLs.
- Checking history page rendering.

All core functionalities worked as expected in the local development environment.

10. Limitations

As a basic version, the application has some limitations:

- No user authentication.
- No handling of short code collisions.
- No click analytics or expiration logic.
- No deployment or production-level configuration.

These limitations are acceptable given the project scope.

11. Future Enhancements

Potential improvements include:

- Custom short URLs.
- User login and dashboard.
- Click tracking and analytics.
- Expiry dates for short URLs.
- REST API support.
- Deployment on cloud platforms.

12. Conclusion

This project demonstrates a clear and structured approach to building a **basic URL shortener** using Flask and SQLAlchemy. The solution emphasizes simplicity, readability, and correct handling of core functionality. Through proper separation of concerns and clean design choices, the application serves as a strong foundation that can be extended into a production-ready system in the future.
